The Stoce Times

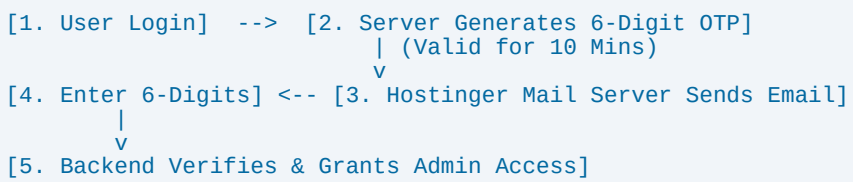
Complete OTP 2FA & Hostinger SMTP Developer Guide

Beginner-Friendly Technical Manual & Source Code Documentation

1. System Architecture & Beginner 5-Step Flow

The OTP system provides 2-Factor Authentication (2FA) for Admin & Author logins. It connects Node.js Express Backend with Hostinger SSL/TLS SMTP Mail Server (smtp.hostinger.com:465) to deliver 6-digit verification codes.

SYSTEM STEP-BY-STEP FLOW DIAGRAM:



2. Step-by-Step Beginner Explanation

Step 1 (Login Request): User submits email & password on Admin Login modal.
Step 2 (Code Generation): Server creates a 6-digit random code: Math.floor(100000 + Math.random() * 900000).
Step 3 (Expiry Logic): Expiry time is set to 10 minutes: Date.now() + 10 * 60 * 1000.
Step 4 (Email Delivery): Nodemailer connects to Hostinger SMTP (smtp.hostinger.com:465) and sends HTML email.
Step 5 (Verification): User enters 6 digits. Server validates code & expiration, then grants Admin access.

3. Environment Variables (.env File Setup)

```
# Server & Database Setup
PORT=5000
DB_HOST=localhost
DB_USER=stoce_user
DB_PASSWORD=YourStrongPassword123!
DB_NAME=finance_pulse_db

# Hostinger SMTP Mail Server Configuration
SMTP_HOST=smtp.hostinger.com
SMTP_PORT=465
SMTP_SECURE=true
SMTP_USERNAME=info@avedatechnologies.com
SMTP_PASSWORD=Jaymatadi@122
SMTP_FROM_EMAIL=info@avedatechnologies.com
SMTP_FROM_NAME=The Stoce Times Security
```

4. Express Server OTP Source Code (server/index.ts)

```
import express from 'express';
import nodemailer from 'nodemailer';

const app = express();
const otpStore: Record<string, { otp: string; expiresAt: number }> = {};

// 1. Hostinger Nodemailer Transporter Setup
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp.hostinger.com',
  port: Number(process.env.SMTP_PORT) || 465,
  secure: true,
  auth: {
    user: process.env.SMTP_USERNAME || 'info@avedatechnologies.com',
    pass: process.env.SMTP_PASSWORD || 'Jaymatadi@122',
  },
});

// 2. Send OTP Endpoint (/api/auth/send-otp)
app.post('/api/auth/send-otp', async (req, res) => {
  const { email } = req.body;
  const otpCode = Math.floor(100000 + Math.random() * 900000).toString();
  const expiresAt = Date.now() + 10 * 60 * 1000; // 10 Minutes

  otpStore[email] = { otp: otpCode, expiresAt };

  await transporter.sendMail({
    from: '"The Stoce Times Security" <info@avedatechnologies.com>',
    to: email,
    subject: '0=Ý  Your Security Verification Code',
    html: `<div style="font-family:sans-serif; background:#f8fafc; padding:20px;">
      <div style="max-width:500px; background:#fff; padding:20px; border-radius:12px;">
        <h2>The Stoce Times 2FA Code</h2>
        <h1 style="color:#16a34a; letter-spacing:6px;">${otpCode}</h1>
        <p>This code is valid for 10 minutes. Do not share it with anyone.</p>
      </div>
    </div>`
  });

  res.json({ success: true, message: 'OTP sent to email.' });
});

// 3. Verify OTP Endpoint (/api/auth/verify-otp)
app.post('/api/auth/verify-otp', (req, res) => {
  const { email, otp } = req.body;
  const record = otpStore[email];

  if (!record) {
    return res.status(400).json({ success: false, message: 'OTP expired or not found.' });
  }

  if (Date.now() > record.expiresAt) {
    delete otpStore[email];
    return res.status(400).json({ success: false, message: 'OTP expired. Request a new one.' });
  }

  if (record.otp !== otp) {
    return res.status(400).json({ success: false, message: 'Invalid OTP code.' });
  }

  delete otpStore[email]; // Clear after successful single use
  res.json({ success: true, message: 'OTP verified successfully.' });
});
```

5. Frontend 2FA React Modal (AdminLoginModal.tsx)

```
// React 6-Digit Auto-Focus OTP Input Handler
const handleDigitChange = (index: number, value: string) => {
  if (!/^[0-9]?$/.test(value)) return;
  const newOtp = [...otpDigits];
  newOtp[index] = value;
  setOtpDigits(newOtp);

  // Auto-focus next input box
  if (value && index < 5) {
    inputRefs.current[index + 1]?.focus();
  }
};

// 10-Minute Live Countdown Timer Effect
useEffect(() => {
  if (step === 'otp' && countdown > 0) {
    const timer = setInterval(() => setCountdown(prev => prev - 1), 1000);
    return () => clearInterval(timer);
  }
}, [step, countdown]);
```

6. Security Rules & Verification Checklist

```
' 1. 6-Digit Random Code: Generated using secure Math.floor(100000 + Math.random() * 900000).
' 2. 10-Minute Expiration: Valid strictly for 600 seconds. Expired codes are rejected automatically.
' 3. Single-Use Invalidation: OTP is deleted from memory (delete otpStore[email]) immediately upon verification.
' 4. Hostinger SSL/TLS Connection: Connects securely on Port 465 (smtp.hostinger.com).
' 5. Real-Time Admin Settings Connection Test: Admin can verify SMTP connection at /api/smtp-config/test.
```